

1.8. Matritsali amallar (Matrix Operations)

NumPy-da matrisa 2D massiv ko'rinishida ifodalanadi. Matrisa amallari chiziqli algebraning asosiy qismidir. Matrisalarni qo'shish, ayirish va ko'paytirish ularni manipulyatsiya qilishda poydevor hisoblanadi. Matrisalar bilan ishlash uchun NumPy oddiy asboblarni taqdim etadi. Masalan, `np.transpose()` qatorlarni ustunlarga va ustunlarni qatorlarga aylantirish orqali matrisani ag'daradi. Agar matrisaning shaklini o'zgartirmoqchi bo'lsangiz (masalan, bitta qatorni bir nechta qatorga aylantirish), `np.reshape()` funksiyasidan foydalanasiz. Matrisani soddalashtirish va uni bitta qiymatlar ro'yxatiga aylantirish uchun `np.flatten()` ishlatilishi mumkin. Nihoyat, `np.dot()` ikki matrisani ko'paytirish uchun qo'llaniladi.

1.8.1. Matritsalarini Qo'shish, Ayirish va Ko'paytirish (Matrix Addition, Subtraction, and Multiplication)

Python-da matrisalar 2D ro'yxatlar yoki 2D massivlar ko'rinishida ifodalanishi mumkin. Matrisalar uchun NumPy massivlaridan foydalanish turli amallarni samarali bajarish uchun qo'shimcha imkoniyatlarni taqdim etadi. NumPy — bu tezkor va optimallashtirilgan massiv amallarini taklif qiluvchi Python kutubxonasidir.

Nima uchun matrisa amallarida NumPy-dan foydalanish kerak?

- **Samarali hisoblash:** Optimallashtirilgan C-darajasidagi implementatsiyalardan foydalanadi.
- **Tozaroq kod:** Ko'plab operatsiyalarda yaqqol sikllarni (loop) yo'qotadi.
- **Keng funktsionallik:** Elementma-element operatsiyalar, matrisalarni ko'paytirish, agregatsiya va boshqalarni qo'llab-quvvatlaydi.

NumPy-da matrisa amallari

1. Elementma-element qo'shish, ayirish va bo'lish

Elementma-element operatsiyalarni bajarish bir xil shakldagi matrisalar o'rtasida arifmetik amallarni to'g'ridan-to'g'ri qo'llash imkonini beradi.

```
import numpy as np

x = np.array([[1, 2], [4, 5]])
y = np.array([[7, 8], [9, 10]])
```

```
print("Qo'shish:\n", np.add(x, y))
print("Ayirish:\n", np.subtract(x, y))
print("Bo'lish:\n", np.divide(x, y))
```

Qo'shish:

```
[[ 8 10]
 [13 15]]
```

Ayirish:

```
[[ -6 -6]
 [ -5 -5]]
```

Bo'lish:

```
[[0.14285714 0.25      ]
 [0.44444444 0.5       ]]
```

Izoh:

- `np.add(x, y)` : Har bir indeksdagi elementlarni mos ravishda qo'shadi (masalan, $1 + 7 = 8$).
- `np.subtract(x, y)` : Birinchi matrisa elementlaridan ikkinchisining mos elementlarini ayiradi.
- `np.divide(x, y)` : Elementlarni bir-biriga bo'ladi. (Eslatma: Yuqoridagi misol natijasida bo'lish amali suzuvchi nuqtali sonlarni qaytaradi).

2. Elementma-element ko'paytirish va Matrisani ko'paytirish

Elementma-element ko'paytirish uchun `np.multiply()` funksiyasidan, standart matrisa ko'paytmasi (chiziqli algebra qoidalari bo'yicha) uchun esa `np.dot()` yoki `@` operatoridan foydalaning.

```
x = np.array([[1, 2], [4, 5]])
y = np.array([[7, 8], [9, 10]])

print("Elementma-element ko'paytirish:\n", np.multiply(x, y))
print("Matrisani ko'paytirish:\n", np.dot(x, y))
```

Elementma-element ko'paytirish:

```
[[ 7 16]
 [36 50]]
```

Matrisani ko'paytirish:

```
[[25 28]
 [73 82]]
```

Izoh:

- **Elementma-element:** Har bir element o'zining mos jufti bilan ko'paytiriladi ($1 \times 7 = 7$, $2 \times 8 = 16$ va h.k.).
- **Matrisa ko'paytmasi:** Birinchi matrisaning qatorlari ikkinchi matrisaning ustunlariga skalyar ko'paytiriladi (masalan, birinchi element: $(1 \times 7) + (2 \times 9) = 25$).

3. Boshqa foydali NumPy matrisa funksiyalari

NumPy kvadrat ildiz chiqarish, yig'indi yoki transponirlash kabi keng tarqalgan amallarni bajarish uchun yordamchi funksiyalarni taqdim etadi.

```
x = np.array([[1, 2], [4, 5]])
y = np.array([[7, 8], [9, 10]])

print("Kvadrat ildiz:\n", np.sqrt(x))
print("Barcha elementlar yig'indisi:", np.sum(y))

print("Ustunlar bo'yicha yig'indi:", np.sum(y, axis=0))
print("Qatorlar bo'yicha yig'indi:", np.sum(y, axis=1))
print("Transponirlash:\n", x.T)
```

Kvadrat ildiz:

```
[[1.          1.41421356]
 [2.          2.23606798]]
```

Barcha elementlar yig'indisi: 34

Ustunlar bo'yicha yig'indi: [16 18]

Qatorlar bo'yicha yig'indi: [15 19]

Transponirlash:

```
[[1 4]
 [2 5]]
```

Izoh:

- `np.sqrt(x)` : Har bir elementdan alohida kvadrat ildiz oladi.
- `np.sum(y, axis=0)` : Ustunlarni (vertikal) qo'shadi.
- `np.sum(y, axis=1)` : Qatorlarni (gorizontal) qo'shadi.
- `x.T` : Matrisani ag'daradi (qatorlar ustunga aylanadi).

Ichma-ich sikllar (Nested Loops) yordamida matrisa amallari

Agar siz NumPy kutubxonasidan foydalanmasangiz, matrisa amallarini ichki `for` sikllari yordamida bajarishingiz mumkin. Bu usulda har bir elementga indekslar orqali murojaat qilinadi:

```

A = [[1, 2], [4, 5]]
B = [[7, 8], [9, 10]]
rows = len(A)
cols = len(A[0])

# Qo'shish uchun bo'sh matrisa yaratish
C = [[0 for i in range(cols)] for j in range(rows)]
for i in range(rows):
    for j in range(cols):
        C[i][j] = A[i][j] + B[i][j]
print("Qo'shish:\n", C)

# Ayirish uchun bo'sh matrisa yaratish
D = [[0 for i in range(cols)] for j in range(rows)]
for i in range(rows):
    for j in range(cols):
        D[i][j] = A[i][j] - B[i][j]
print("Ayirish:\n", D)

# Bo'lish uchun bo'sh matrisa yaratish
E = [[0 for i in range(cols)] for j in range(rows)]
for i in range(rows):
    for j in range(cols):
        E[i][j] = A[i][j] / B[i][j]
print("Bo'lish:\n", E)

```

Qo'shish:

```
[[8, 10], [13, 15]]
```

Ayirish:

```
[[ -6, -6], [-5, -5]]
```

Bo'lish:

```
[[0.14285714285714285, 0.25], [0.4444444444444444, 0.5]]
```

Izoh: Ushbu usulda biz avval natijani saqlash uchun nol bilan to'ldirilgan matrisa yaratib olamiz. So'ngra tashqi sikl qatorlar bo'ylab (`i`), ichki sikl esa ustunlar bo'ylab (`j`) harakatlanib, mos elementlarni birma-bir hisoblaydi. Bu usul NumPy-ga qaraganda ancha sekin ishlaydi va ko'p kod yozishni talab qiladi.

1.8.2. Matritsani transponirlash (Matrix Transpose)

NumPy-dagi `matrix.transpose()` metodi matrisani transponirlash uchun ishlatiladi, ya'ni u matrisani uning diagonal bo'ylab ag'darib, qatorlarni ustunlarga va ustunlarni qatorlarga aylantiradi.

Misol: Kirish: `[[1, 2], [3, 4]]` Natija: `[[1, 3], [2, 4]]`

Sintaksis

```
matrix.transpose()
```

Qaytaradi: Asl matrisaning transponirlangan varianti bo'lgan yangi matrisa.

1-misol: Ushbu misol 2×3 o'lchamli matrisa yaratadi va `transpose()` metodi yordamida uning transponirlangan shaklini topadi.

```
# 2x3 o'lchamli matrisa yaratamiz
a = np.matrix([[1, 2, 3], [4, 5, 6]])
b = a.transpose()

print("Transponirlangan matrisa:")
print(b)
```

Transponirlangan matrisa:

```
[[1 4]
 [2 5]
 [3 6]]
```

Izoh: Asl matrisada birinchi qator `[1, 2, 3]` edi, transponirlashdan so'ng u birinchi ustun bo'lib qoldi. Ikkinchi qator `[4, 5, 6]` esa ikkinchi ustunga aylandi. Natijada matrisa o'lchami 2×3 dan 3×2 ga o'zgardi. Shuningdek, NumPy-da bu amalni qisqacha `a.T` ko'rinishida ham bajarish mumkin.

2-misol: Bu yerda 3×3 o'lchamli matrisa yaratilgan va xuddi shu metod yordamida transponirlangan.

```
# Matrisani satr ko'rinishida e'lon qilish
a = np.matrix('[4, 1, 9; 12, 3, 1; 4, 5, 6]')
b = a.transpose()

print(b)
```

```
[[ 4 12  4]
 [ 1  3  5]
 [ 9  1  6]]
```

Izoh: Matrisaning bosh diagonal (`4, 3, 6`) o'z o'rnida qoladi, qolgan elementlar esa diagonalga nisbatan o'rin almashadi. Masalan, birinchi ustun (`4, 12, 4`) transponirlashdan so'ng birinchi qatorga aylanadi.

3-misol: Matrisalarni ko'paytirishda transponirlashdan foydalanish.

```

a = np.matrix([[1, 2], [3, 4]])
b = np.matrix([[5, 6], [7, 8]])

# b matrisani transponirlab, keyin a matrisaga ko'paytirish
res = a * b.transpose()
print(res)

[[17 23]
 [39 53]]

```

Izoh: Ushbu amalda avval `b` matrisasi transponirlanadi (ya'ni `[[5, 7], [6, 8]]` holatiga keladi), so'ngra `a` matrisasiga ko'paytiriladi. Matrisalar ko'paytmasida transponirlashdan foydalanish chiziqli regressiya va neyron tarmoqlar kabi sohalarda og'irlik koeffitsientlarini (weights) hisoblashda juda ko'p qo'llaniladi.

1.8.3. Matritsa determinanti va teskari matritsa (Determinant and Inverse of a Matrix)

Matrisani teskarilash — bu ko'paytirish jarayonida boshqa bir matrisaning ta'sirini bekor qiladigan (teskari bo'lgan) matrisani topish jarayonidir. NumPy matrisaning teskarisini hisoblash uchun samarali funksiyalarni taqdim etadi, bu esa tenglamalar tizimini yechish va chiziqli algebra amallarini bajarishni osonlashtiradi.

Matrisaning teskarisi faqat u **maxsus bo'lmagan** (non-singular) bo'lgandagina mavjud bo'ladi, ya'ni uning determinanti 0 ga teng bo'lmasligi kerak. Determinant va biriktirilgan (adjoint) matrisa yordamida kvadrat matrisaning teskarisini quyidagi formula orqali osongina topishimiz mumkin:

Agar $\det(A) \neq 0$ bo'lsa:

$$A^{-1} = \frac{\text{adj}(A)}{\det(A)}$$

Aks holda: "Teskari matrisa mavjud emas"

Matrisa tenglamasi:

Matrisa tenglamalarini yechishda teskari matrisa quyidagicha qo'llaniladi:

$$Ax = B \implies A^{-1}Ax = A^{-1}B \implies x = A^{-1}B$$

Bu yerda:

- A^{-1} : A matrisasining teskarisi

- x : Noma'lum o'zgaruvchilar ustuni
- B : Yechimlar matrisasi (ozod hadlar)

Izoh: Amaliyotda, masalan, neyron tarmoqlarda yoki muhandislik hisob-kitoblarida $Ax = B$ ko'rinishidagi chiziqli tenglamalar tizimini yechish uchun ko'pincha A matrisasining teskarisini topish talab etiladi. NumPy-da bu amalni `np.linalg.inv(A)` funksiyasi yordamida bir necha millisoniyalarda bajarish mumkin.

NumPy yordamida teskari matrisani topish

Python-da teskari matrisani hisoblash uchun NumPy modulining `numpy.linalg.inv()` funksiyasidan foydalaniladi.

Sintaksis: `numpy.linalg.inv(a)`

Parametrlar:

- **a:** Teskarisi hisoblanishi kerak bo'lgan matrisa (kvadrat matrisa bo'lishi shart).

Qaytaradi: `a` matrisasining teskari shakli.

1-misol: Ushbu misolda 3×3 o'lchamli NumPy massivi yaratiladi va `np.linalg.inv()` yordamida uning teskarisi topiladi.

```
A = np.array([[6, 1, 1],
              [4, -2, 5],
              [2, 8, 7]])
```

```
print(np.linalg.inv(A))
```

```
[[ 0.17647059 -0.00326797 -0.02287582]
 [ 0.05882353 -0.13071895  0.08496732]
 [-0.11764706  0.1503268   0.05228758]]
```

2-misol: Ushbu misolda 4×4 o'lchamli NumPy massivi yaratiladi va uning teskarisi hisoblanadi.

```
A = np.array([[6, 1, 1, 3],
              [4, -2, 5, 1],
              [2, 8, 7, 6],
              [3, 1, 9, 7]])
```

```
print(np.linalg.inv(A))
```

```
[ [ 0.13368984  0.10695187  0.02139037 -0.09090909]
  [-0.00229183  0.02673797  0.14820474 -0.12987013]
  [-0.12987013  0.18181818  0.06493506 -0.02597403]
  [ 0.11000764 -0.28342246 -0.11382735  0.23376623]]
```

Izoh: Matrisa o'lchami kattalashgani sayin uni qo'lda hisoblash (determinant va adjoint orqali) juda murakkablashib ketadi. `np.linalg.inv()` funksiyasi esa murakkablikdan qat'i nazar, agar teskari matrisa mavjud bo'lsa (determinant 0 bo'lmasa), natijani tezda hisoblab beradi. Agar matrisa "maxsus" (singular) bo'lsa, ya'ni teskarisi bo'lmasa, NumPy `LinAlgError` xatoligini qaytaradi.

3-misol: Ushbu misolda `np.linalg.inv()` funksiyasi yordamida bir vaqtning o'zida bir nechta NumPy matrisalarining (matrisalar to'plamining) teskarisini hisoblash ko'rsatilgan.

```
# 3D massiv: ikkita 2x2 o'lchamli matrisadan iborat to'plam
A = np.array([[ [1., 2.], [3., 4.]],
               [[1, 3], [3, 5]]])

print(np.linalg.inv(A))
```

```
[ [-2.   1.  ]
  [ 1.5 -0.5 ]]

[ [-1.25  0.75]
  [ 0.75 -0.25]]]
```

Izoh: NumPy-ning kuchi shundaki, u nafaqat bitta matrisa bilan, balki matrisalar "stecki" (3D massivlar) bilan ham ishlay oladi. Bu yerda `A` massivi ikkita 2×2 matrisani o'z ichiga olgan. `np.linalg.inv(A)` funksiyasi har bir matrisaning teskarisini alohida-alohida hisoblab, natijani yana xuddi shunday tuzilmada qaytaradi. Bu xususiyat mashinali o'qitishda bir vaqtning o'zida ko'plab ma'lumotlar bloklarini (batches) qayta ishlashda juda qo'l keladi.

1.8.4. Vektorlarning skalyar ko'paytmasini hisoblash (Dot Product of Two Vectors)

`numpy.ndarray.dot()` funksiyasi ikki massivning **skalyar ko'paytmasini** (dot product) hisoblaydi. U chiziqli algebra, mashinali o'qitish va chuqur o'qitish (deep learning) sohalarida matrisalarni ko'paytirish va vektor proyeksiyalari kabi amallar uchun keng qo'llaniladi.

Misol:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = np.dot(a, b)
print(result)
```

32

Skalyar ko'paytmani tushunish

- Agar **ikkala kirish ham 1D massiv** bo'lsa, `dot()` ichki ko'paytmani (inner product) hisoblaydi va natija skalyar (bitta son) bo'ladi.
- Agar **kirishlardan biri N-o'lchamli massiv** bo'lsa, `dot()` matrisalarni ko'paytirish amalini bajaradi.
- Agar **kirishlardan biri skalyar** bo'lsa, `dot()` elementma-element ko'paytirishni amalga oshiradi.

Sintaksis

```
numpy.ndarray.dot(arr, out=None)
```

Parametrlar:

- `arr` (array_like): Skalyar ko'paytma uchun kirish massivi.
- `out` (ndarray, ixtiyoriy): Natijani saqlash uchun mo'ljallangan argument.

Qaytaradi: Kirish shakliga qarab **skalyar**, **vektor** yoki **matrisa**.

Izoh: Yuqoridagi 1D misolda hisob-kitob quyidagicha amalga oshadi:
 $(1 \times 4) + (2 \times 5) + (3 \times 6) = 4 + 10 + 18 = 32$. Matrisalar bilan ishlaganda esa birinchi matrisaning qatorlari ikkinchi matrisaning ustunlariga mos ravishda ko'paytirilib, yig'iladi.

Kod implementatsiyasi

1-misol: Matrisalarni ko'paytirish uchun `numpy.ndarray.dot()` dan foydalanish

```
# 3x3 o'lchamli birlik matrisa (identity matrix) yaratamiz
arr1 = np.eye(3)

# 3x3 o'lchamli, barcha elementlari 3 ga teng bo'lgan matrisa yaratamiz
arr = np.ones((3, 3)) * 3
```

```
# Dot product (ko'paytma) hisoblaymiz
gfg = arr1.dot(arr)

print(gfg)
```

```
[[3. 3. 3.]
 [3. 3. 3.]
 [3. 3. 3.]]
```

Izoh: Birlik matrisa (`np.eye`) har qanday matrisaga ko'paytirilganda, o'sha matrisaning o'zi natija sifatida chiqadi. Shuning uchun barcha elementlari 3 bo'lgan matrisa o'zgarishsiz qoldi.

2-misol: Ketma-ket bir nechta skalyar ko'paytmalarni bajarish

```
arr1 = np.eye(3)
arr = np.ones((3, 3)) * 3

# Ketma-ket ko'paytirish: (arr1 * arr) * arr
gfg = arr1.dot(arr).dot(arr)

print(gfg)
```

```
[[27. 27. 27.]
 [27. 27. 27.]
 [27. 27. 27.]]
```

Izoh: Bu yerda birinchi ko'paytma natijasi (barcha elementlari 3 bo'lgan matrisa) yana barcha elementlari 3 bo'lgan matrisaga ko'paytirilmoqda. Matrisa ko'paytirish qoidasiga ko'ra, har bir katakda $3 \times 3 + 3 \times 3 + 3 \times 3 = 27$ amali bajariladi.

1.8.5. Matritsa dispersiyasini hisoblash (Finding Variance of a Matrix)

`numpy.matrix.var()` metodi yordamida biz matrisaning dispersiyasini (variance) topishimiz mumkin. Dispersiya ma'lumotlarning o'rtacha qiymatdan qanchalik darajada tarqalganligini ko'rsatadigan statistik o'lchovdir.

Sintaksis: `matrix.var()` **Qaytaradi:** Matrisaning dispersiya qiymatini.

1-misol: Ushbu misolda `matrix.var()` metodi berilgan matrisaning dispersiyasini qanday topishini ko'rib chiqamiz.

```
# Numpy yordamida matrisa yaratamiz
gfg = np.matrix('[4, 1; 12, 3]')
```

```
# matrix.var() metodini qo'llaymiz
geek = gfg.var()

print(geek)
```

17.5

Izoh: Dispersiya hisoblanayotganda avval barcha elementlarning o'rtacha qiymati topiladi ($((4 + 1 + 12 + 3)/4 = 5)$), so'ngra har bir elementning o'rtacha qiymatdan farqi kvadratga ko'tarilib, ularning o'rtachasi hisoblanadi.

2-misol:

```
# Numpy yordamida 3x3 matrisa yaratamiz
gfg = np.matrix('[4, 1, 9; 12, 3, 1; 4, 5, 6]')

# matrix.var() metodini qo'llaymiz
geek = gfg.var()

print(geek)
```

11.555555555555555

Izoh: Bu yerda dispersiya matrisadagi barcha 9 ta element uchun hisoblangan. Agar sizga faqat qatorlar yoki ustunlar bo'yicha dispersiya kerak bo'lsa, metod ichida `axis` parametridan (`axis=0` yoki `axis=1`) foydalanishingiz mumkin.



Nazorat savollari

1-daraja: Asosiy tushunchalar

1. NumPy-da `np.matrix()` funksiyasining asosiy vazifasi nima va u oddiy massivlardan qanday farq qiladi?
2. Matritsaning o'lchamini aniqlash uchun qaysi atributdan foydalaniladi?
3. Matritsaning bosh diagonalidagi elementlarni ajratib olish uchun qaysi metod ishlatiladi?
4. Matritsali amallarda dispersiya (`.var()`) va standart og'ish (`.std()`) nima uchun hisoblanadi?

2-daraja: Funksiyalarning ishlash mexanizmi

5. `.getA()` metodi matritsa obyekti ustida qo'llanilganda u qanday turdagi ma'lumotni qaytaradi?

6. Matritsaning determinanti (`np.linalg.det()`) nima va u qanday turdagi matritsalar uchun hisoblanadi?
7. Chiziqli tenglamalar sistemasini yechishda `np.linalg.solve()` funksiyasining ishlash tartibini tushuntiring.
8. Matritsaning oʻrta qiymatini (`.mean()`) qatorlar yoki ustunlar boʻyicha alohida hisoblash qanday amalga oshiriladi?

3-daraja: Amaliy tahlil va murakkab amallar

9. Matritsani transponirlash (`.transpose()`) amali uning tuzilishini qanday oʻzgartiradi?
10. Teskari matritsani (`np.linalg.inv()`) topish jarayonida determinant qiymati qanday ahamiyatga ega?
11. `np.linalg.eig()` funksiyasi qaytaradigan xos qiymatlar (eigenvalues) va xos vektorlar (eigenvectors) amaliyotda qayerda qoʻllaniladi?
12. Matritsaning yigʻindisini (`.sum()`) hisoblashda `axis` parametrining roli va ahamiyatini izohlang.